PA3 - Sistema de gestión de tareas

Trabajo integrador de Desarrollo Web Full Stack

Estudiante: Justin Leonardo Cordero Velito
Fecha: 9 de noviembre de 2026
Proyecto: Organiza+ (gestión de tareas)

1. Descripción del proyecto

Organiza+ es una aplicación para llevar un control de tareas. Permite registrar actividades, cambiar su estado, colocar una fecha de vencimiento y eliminar las que ya no se necesitan. La idea es tener la información ordenada y que cada usuario pueda revisar sus propias tareas.

El sistema tiene registro de usuarios, inicio de sesión, permisos y dos paneles para trabajar con las tareas. La interfaz principal está desarrollada en Angular y el panel secundario en React. Ambos utilizan la misma API, asi que la información queda guardada en un solo lugar y no depende de la pantalla que se use.

2. Tecnologías utilizadas

Tecnología	Para qué se utiliza
Angular 16 y TypeScript	Interfaz principal, dashboard, formularios, navegación y consumo de la API.
React 18 y TypeScript	Panel secundario con Hooks, rutas y estado compartido por Context API.
Node.js y Express	Servidor HTTP con rutas REST, controladores y validación de solicitudes.
MongoDB y Mongoose	Guardar las tareas y los usuarios de forma persistente.
bcrypt y JWT	Proteger contraseñas y controlar las sesiones.
Docker	Ejecutar los cuatro servicios desde una sola configuración.

3. Funcionamiento general

El navegador se comunica con Express utilizando peticiones HTTP y respuestas en JSON. Express valida los datos, revisa el token cuando corresponde y consulta MongoDB por medio de Mongoose. El trabajo de autenticación y permisos queda en el backend, no solamente en las pantallas.

Cliente	API	Base de datos
Angular 16 Interfaz principal	Express / Node.js Rutas: auth, tasks, admin	MongoDB users y tasks
React 18 Panel secundario	Misma API REST y mismos permisos	Mismas colecciones

4. Organización del proyecto y modelo de datos

El código está separado en tres carpetas principales: **angular** para la interfaz, **react** para el panel adicional y **backend** para el servidor. La carpeta **docs** contiene la arquitectura, los endpoints, las pruebas y los pasos de despliegue. También se incluye el archivo **CHANGELOG.md** con las modificaciones del proyecto.

Colección	Campos	Uso
users	_id, name, email, passwordHash, role, tokenVersion	Datos de la cuenta y control de permisos. El correo es único.
tasks	_id, title, description, status, dueDate, owner	Datos de cada tarea. owner indica a qué usuario pertenece.

El campo **status** puede tener tres valores: pendiente, en_progreso y completada. Los esquemas también guardan fechas de creación y modificación. La contraseña no se almacena como texto normal, sino como un hash.

5. Rutas principales de la API

Método	Ruta	Función
POST	/api/auth/register	Crear una cuenta normal.
POST	/api/auth/login	Iniciar sesión y recibir un JWT.
GET / POST	/api/auth/me · /api/auth/logout	Consultar la cuenta o cerrar sesión.
GET / POST	/api/tasks	Listar o registrar tareas.
GET	/api/tasks/summary	Consultar cantidades por estado.
GET / PUT / PATCH / DELETE	/api/tasks/:id	Ver, editar o eliminar una tarea.
GET	/api/admin/users	Ver usuarios, solo administrador.
GET	/api/admin/metrics	Consultar cifras generales, solo administrador.

La API responde con códigos HTTP según cada caso. Por ejemplo, 201 al crear un registro, 400 cuando un dato es inválido, 401 si no existe una sesión válida, 403 cuando el rol no tiene permiso y 404 si no se encuentra la tarea. Los detalles de las rutas están en **docs/ENDPOINTS.md**.

6. Desarrollo de las funciones solicitadas

JavaScript y TypeScript (T01-T02). Se utilizaron const, funciones flecha, métodos de arreglos, módulos, promesas y async/await. TypeScript se usa en los clientes y en el servidor, con interfaces para los datos, tipos reutilizables y enumeraciones para roles y estados.

Angular (T03-T04). La parte principal tiene dashboard, listado de tareas y formularios para crear o editar. Los formularios son reactivos, con validaciones de campos. Para enviar información se utiliza HttpClient y un interceptor que agrega el token. Las rutas protegidas evitan entrar a una página privada sin iniciar sesión.

Backend y MongoDB (T05-T07). Express organiza las rutas, valida la información recibida y maneja los errores. Mongoose define los modelos y guarda las operaciones CRUD. En el registro de usuarios se usa bcrypt y, al iniciar sesión, el servidor entrega un JWT. Se comprueba el rol y también el dueño de cada tarea.

React y estado global (T09-T11). El panel de React tiene componentes funcionales, React Router DOM, Hooks y Context API para compartir el usuario y la sesión. Las peticiones se hacen con Fetch. Si el token vence o el servidor devuelve 401, se solicita iniciar sesión nuevamente.

Bitácora (T08). En CHANGELOG.md se resumen las funciones de gestión de tareas, los ajustes del backend, la autenticación, los permisos y el panel React.

7. Seguridad

Las contraseñas usan bcrypt y los JWT tienen una duración de una hora. El servidor valida la firma del token y revisa la versión de sesión almacenada en la base de datos. Al cerrar sesión, la versión cambia y los tokens anteriores dejan de servir.

El usuario normal solo puede trabajar con sus propias tareas. El administrador puede ver todas y consultar el panel de usuarios. Tambien se aplican validaciones en el servidor, limitación de solicitudes al iniciar sesión, CORS y cabeceras con Helmet. Las claves se configuran mediante variables de entorno y no se colocan directamente en el código.

8. Pruebas del sistema

Se incluyeron pruebas de funciones del backend y un script que realiza peticiones HTTP a la API. Para revisar el funcionamiento completo se pueden seguir estos casos:

Prueba	Acción	Resultado esperado
Cuenta	Registrar e iniciar sesión.	Se obtiene el perfil del usuario y su token.
CRUD	Crear, editar, listar y eliminar una tarea.	Los cambios se guardan en MongoDB.
Dos interfaces	Crear una tarea en Angular y abrir React.	Aparece la tarea al actualizar el listado.
Permisos	Entrar a /api/admin/users con rol user.	El servidor devuelve 403.
Cierre de sesión	Cerrar sesión e intentar usar el mismo JWT.	El servidor responde 401.
Persistencia	Reiniciar los servicios y consultar tareas.	Se conservan los registros no eliminados.

En **docs/PRUEBAS_Y_EVIDENCIAS.md** está la lista para revisar las funciones y guardar las capturas que se piden en la entrega.

9. Ejecución y publicación

Para abrir el proyecto en Windows, primero se inicia Docker Desktop. Luego se abre PowerShell en la carpeta principal y se ejecuta **.\INICIAR_WINDOWS.ps1**. También se puede hacer manualmente: copiar .env.example como .env, poner una clave en JWT_SECRET y ejecutar este comando:

docker compose up --build -d

Angular se abre en **localhost:4200**, React en **localhost:5173** y la API se puede consultar en **localhost:4000/api/health**. Se usa la misma cuenta en las dos páginas porque las tareas se guardan en una sola base de datos.

Para subirlo a internet se pueden utilizar los Dockerfile que están en cada carpeta y una base de datos de MongoDB Atlas. Los pasos están en **docs/DESPLIEGUE.md**.

10. Conclusión

Organiza+ reúne las funciones principales para gestionar tareas y controlar quién puede acceder a ellas. Angular y React trabajan con la misma API, asi que la información se mantiene en un solo lugar.

Referencias técnicas

Angular, compatibilidad de versiones: https://angular.dev/reference/versions

Express, seguridad: https://expressjs.com/advanced/best-practice-security/

Mongoose, validaciones: https://mongoosejs.com/docs/validation.html